MEDISYNC

MEDICINE REMINDER & STOCK MONITORING SYSTEM

PROJECT REPORT

Submitted in partial fulfilment of the academic requirements
for the Bachelor of Technology programme

Submitted by

Swagatam Behera (AP25110070661)

Harsh Kumar (AP25110070658)

Muskan Singh (AP25110070440)

Katragadda Neha Sri (AP25110070846)

P Parwish (AP25110070650)

B.Tech – Computer Science and Engineering
(Artificial Intelligence & Machine Learning)

Under the Guidance of

Prof. Sikhinam Nagamani

Department of Computer Science and Engineering
SRM University–AP

Academic Year 2026–2027

November 2026

Declaration

We, the team members, declare that the project entitled “MediSync – Medicine Reminder & Stock Monitoring System” is submitted as part of my academic work for the Bachelor of Technology programme at SRM University–AP. This report describes the design, implementation and observed behaviour based on the project materials and screenshots supplied for documentation.

We acknowledge the use of Java, Swing, JDBC, MySQL and Maven. External material used for technical reference should be acknowledged according to institutional academic guidelines.

Date: _____

Acknowledgement

We express our sincere gratitude to our faculty guide, prof.Sikhinam Nagamani, for guidance and academic support during the development and documentation of this project. we also thank the Department of Computer Science and Engineering, SRM University–AP, for providing the academic environment and resources required to complete this work.

Abstract

MediSync is a Java-based desktop application developed to organise medicine inventory information and scheduled medicine reminders in a single interface. It provides medicine record management, stock-level monitoring, expiry-date monitoring, reminder scheduling, notification pop-ups and a history view for recording reminder outcomes. The application uses Java Swing for its graphical user interface, JDBC for database access, MySQL for persistent storage and Maven for project build management.

The database contains two related tables, `medicines` and `reminders`. Each reminder references a medicine through a foreign key. Reminder records use PENDING, NOTIFIED, TAKEN, MISSED and SKIPPED statuses. The supplied screenshots show ten medicine records, four low-stock records in the dashboard summary, a due reminder notification and three TAKEN outcomes in reminder history. MISSED and SKIPPED are permitted by the schema but are not demonstrated in the supplied history screenshot.

Keywords: Java Swing, JDBC, MySQL, medicine inventory, stock monitoring, expiry alerts, reminders.

Contents

1 Identification of the Project **1**

1.1 Technology Stack 1

1.2 Scope 1

2 Project Background **2**

3 Problem Statement **3**

4 Objectives of the Project **4**

5 System Design **5**

5.1 Architecture 5

5.2 Modules 5

5.3 Database Relationship 6

5.4 Reminder Status Model 6

6 Implementation of OOP Concepts **7**

7 Implementation of Advanced Java Concepts **8**

7.1 Swing Event Handling 8

7.2 Timer-Based Reminder Checking 8

7.3 JDBC and Maven 8

7.4 Date and Time 8

8 Graphical User Interface (GUI) **9**

9 Database Connectivity Using JDBC **12**

9.1 Database Configuration 12

9.2 Medicines Table 12

9.3 Reminders Table 12

10 Source Code and Output Screenshots **14**

11 Testing and Results **15**

12 Conclusion and Future Enhancements **18**

A Reminder Workflow **19**

B Demonstration Sequence **22**

List of Figures

5.1 MediSync system architecture showing the interaction between the user interface, application data-access components, JDBC and MySQL. 5

8.1 Dashboard summary: 10 medicines, 4 low-stock records, 0 expiring/expired records and 0 active reminders. 9

8.2 Medicine inventory table showing ten medicine records. 10

8.3 Stock and expiry alert view; four records are labelled LOW STOCK. 10

8.4 Due-reminder notification dialog for Paracetamol. 11

8.5 Reminder history with three TAKEN outcomes. 11

9.1 MySQL Workbench result grid for reminder records. 13

9.2 MySQL Workbench result grid for medicine records. 13

A.1 MediSync reminder workflow from scheduling through notification and user-recorded outcome. 20

1. Identification of the Project

Project title: MediSync – Medicine Reminder & Stock Monitoring System
Student: Swagatam Behera
Registration number: AP25110070661
Programme: B.Tech – CSE (Artificial Intelligence & Machine Learning)
Institution: SRM University–AP
Department: Department of Computer Science and Engineering
Faculty guide: Sikhinam Nagamani
Academic year: 2026–2027

1.1 Technology Stack

The project uses Java 17 or later, Java Swing, JDBC, MySQL 8.0.46, MySQL Connector/J 9.4.0 and Apache Maven. The official Java and Swing documentation provides the platform and GUI reference material used for implementation [1, 2]. The GUI entry point is `gui.MediSyncGUI`. The project context identifies model and DAO components including `Medicine`, `Reminder`, `MedicineDAO`, `ReminderDAO` and `DBConnection`.

1.2 Scope

The application manages local medicine inventory records and reminder information. It is not a diagnosis or prescription system and does not independently verify medicine consumption.

2. Project Background

Medicine inventories contain products with different quantities and expiry dates. Maintaining these details manually can make it difficult to identify low-stock items, monitor expiry dates and organise reminder information. MediSync demonstrates how a Java desktop application can combine graphical interaction, object-oriented program organisation and persistent relational storage.

The intended user can maintain medicine records, review stock thresholds, inspect expiry information, schedule reminders and view reminder history. The application requires the database service and desktop process to be running for its local reminder workflow. Cloud synchronisation, mobile notifications and notification delivery after application closure are not established features.

3. Problem Statement

The project addresses the need for a desktop system that stores medicine and reminder records in a relational database, displays inventory information through a graphical interface, identifies records meeting configured stock or expiry conditions, and supports scheduled reminder notifications with user-recorded outcomes.

The main functional problem areas are medicine record management, stock visibility, expiry visibility, reminder scheduling and outcome tracking. The proposed solution combines these functions in a Java Swing application backed by MySQL through JDBC.

4. Objectives of the Project

- Develop a desktop GUI using Java Swing.
- Store medicine details in a MySQL relational database.
- Support medicine record creation, search, editing and deletion.
- Compare medicine quantity with a configured low-stock threshold.
- Provide expiry-related filtering and alert information.
- Schedule reminders against selected medicine records.
- Display an in-application notification when a reminder becomes due while the application is running.
- Record reminder outcomes and retain reminder history.
- Use JDBC and parameterised database operations for persistence.

5. System Design

5.1 Architecture

The Swing presentation layer accepts user input and displays forms, tables, alerts and reminder dialogs. GUI actions call data-access objects, which execute SQL operations through JDBC. The DBConnection component provides connection configuration. MySQL stores medicine and reminder records.

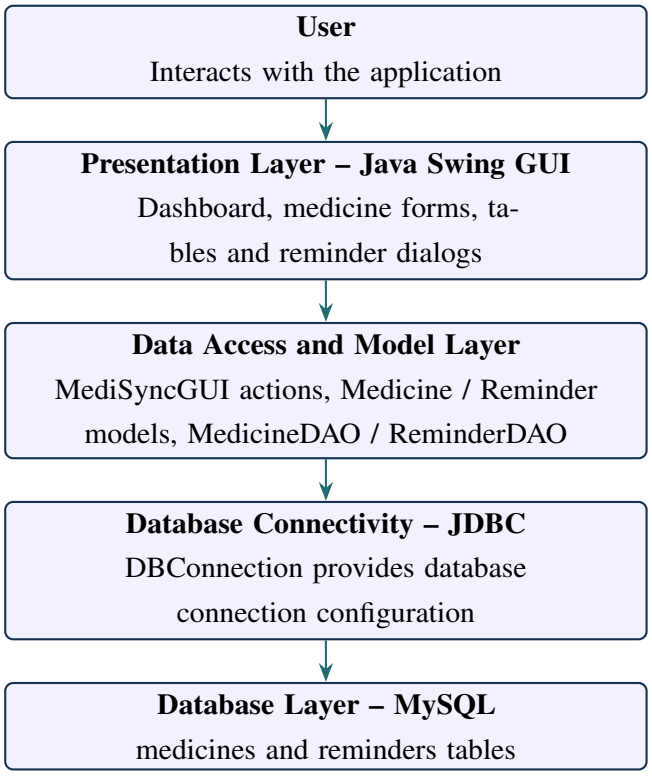


Figure 5.1: MediSync system architecture showing the interaction between the user interface, application data-access components, JDBC and MySQL.

5.2 Modules

Dashboard: summary indicators for total medicines, low stock, expiring/expired records and active reminders. The captured dashboard shows 10, 4, 0 and 0 respectively.

Medicine Management: search, show, add, edit, update stock and delete actions. The

table displays ID, name, type, dosage, stock, low threshold, expiry and notes.

Stock & Expiry: filters for all medicines, low stock, expired and expiring within 30 days. Four records are labelled LOW STOCK in the captured view.

Reminders: selection of a medicine, date/time entry, scheduling and outcome recording. The supplied screenshot shows a due notification for Paracetamol.

History: displays reminder records. The supplied screenshot shows three TAKEN records.

5.3 Database Relationship

The `medicines.medicine_id` column is the primary key. `reminders.medicine_id` is a non-null foreign key referencing it with ON DELETE CASCADE. One medicine can have multiple reminders.

5.4 Reminder Status Model

The schema permits PENDING, NOTIFIED, TAKEN, MISSED and SKIPPED. PENDING represents a scheduled reminder; NOTIFIED represents a triggered notification; the final outcomes are selected by the user. A notification is not proof of consumption.

6. Implementation of OOP Concepts

The project separates domain data, database operations and user-interface responsibilities. `Medicine` and `Reminder` represent domain entities. DAO classes centralise persistence operations and abstract SQL details from the GUI. `DBConnection` abstracts connection creation. Swing itself uses framework inheritance; custom application-specific inheritance and polymorphism should be verified against the source code before being claimed.

7. Implementation of Advanced Java Concepts

7.1 Swing Event Handling

The interface responds to button clicks, tab navigation, row selection and dialog confirmation through event listeners.

7.2 Timer-Based Reminder Checking

The project context indicates that a `javax.swing.Timer` checks for due reminders at a five-second interval; Swing timer behaviour is documented in the Java API [4]. Swing timer listeners execute on the Event Dispatch Thread. Database operations in that listener should be short or moved to a background worker to avoid blocking the interface.

7.3 JDBC and Maven

JDBC connects Java to MySQL [3]. Parameterised statements and try-with-resources are recommended for safe SQL execution and resource cleanup [5]. Maven manages dependencies and the build through `pom.xml` [8]. The earlier build issue was an XML parsing error in the POM, not a Java compilation error.

7.4 Date and Time

Reminder scheduling depends on comparing stored DATETIME values with the application or database clock. Consistent timezone configuration is important when interpreting scheduled and notified timestamps.

8. Graphical User Interface (GUI)

The GUI uses a light workspace, dark navigation panel and teal action controls. The following figures are genuine screenshots supplied for this report.

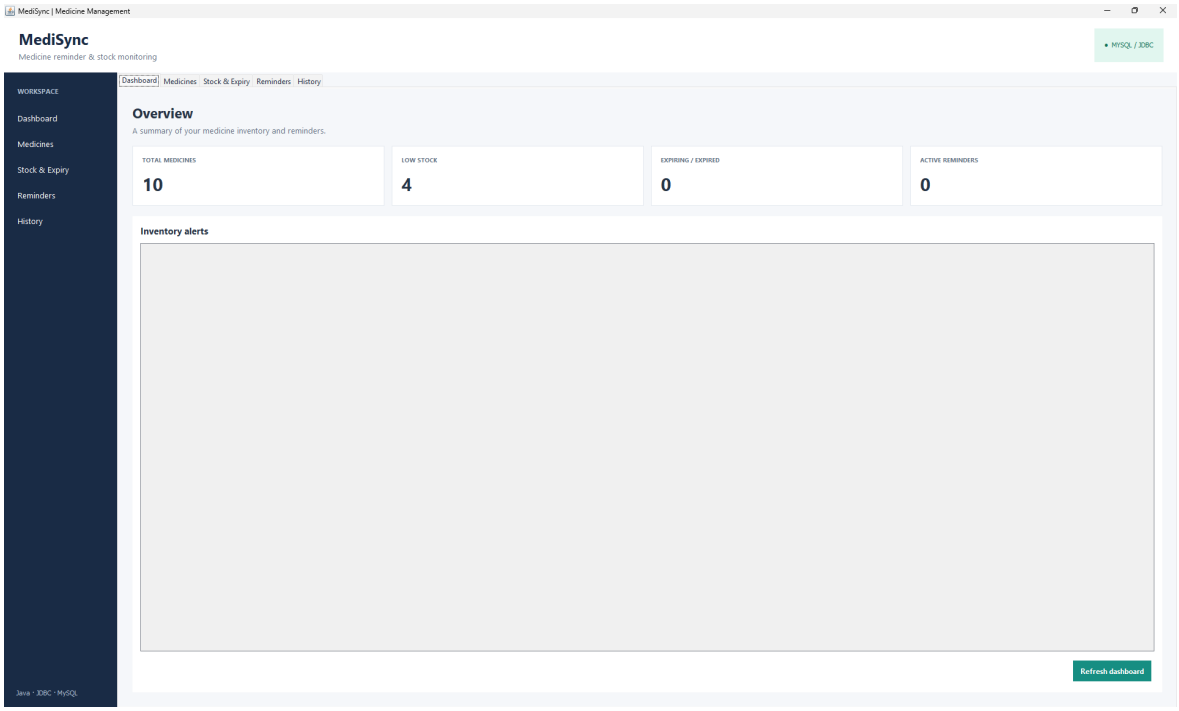


Figure 8.1: Dashboard summary: 10 medicines, 4 low-stock records, 0 expiring/expired records and 0 active reminders.

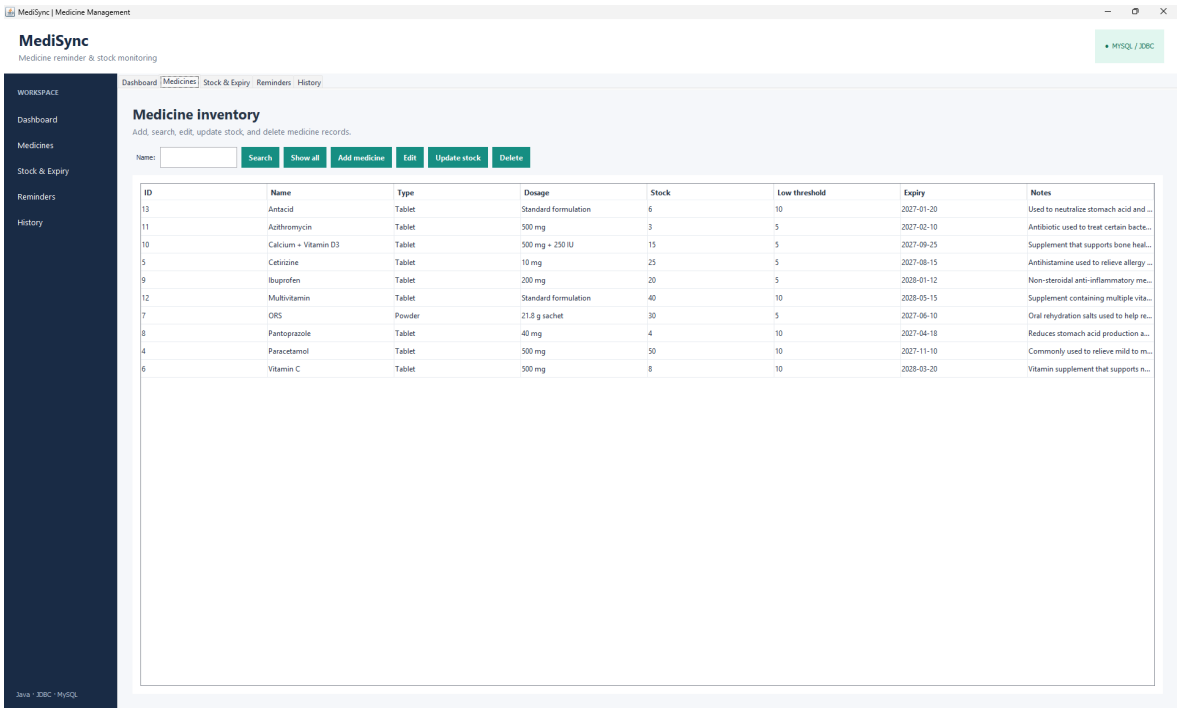


Figure 8.2: Medicine inventory table showing ten medicine records.

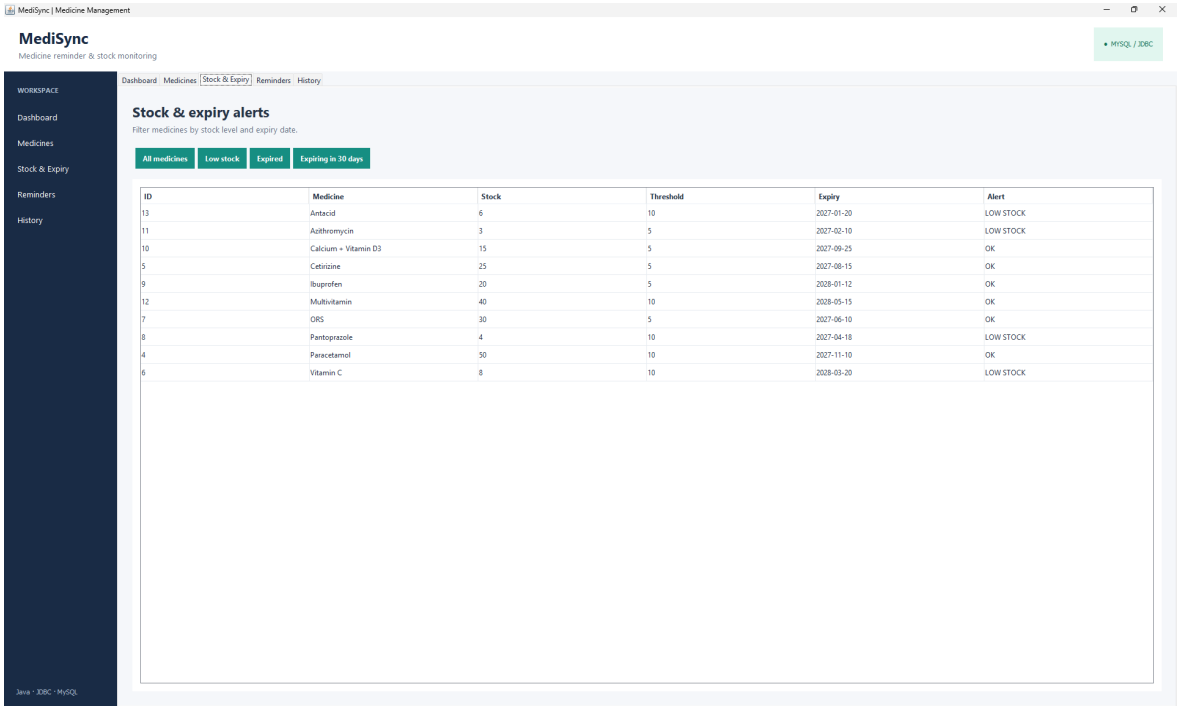


Figure 8.3: Stock and expiry alert view; four records are labelled LOW STOCK.

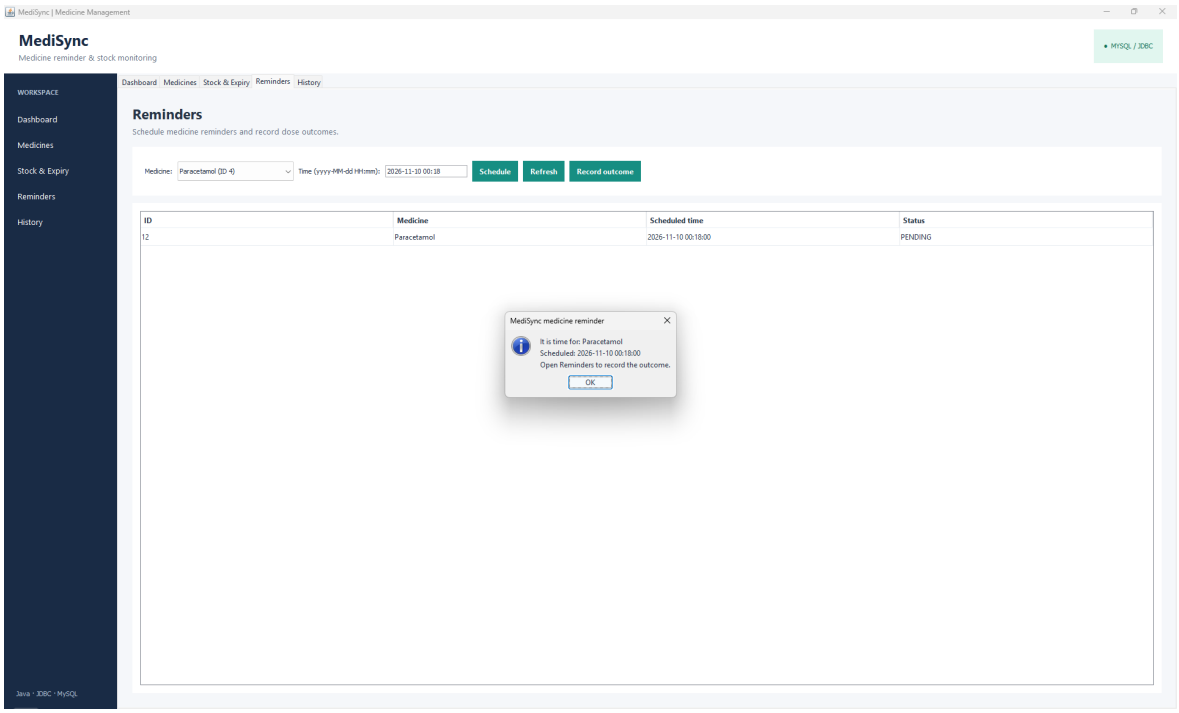


Figure 8.4: Due-reminder notification dialog for Paracetamol.

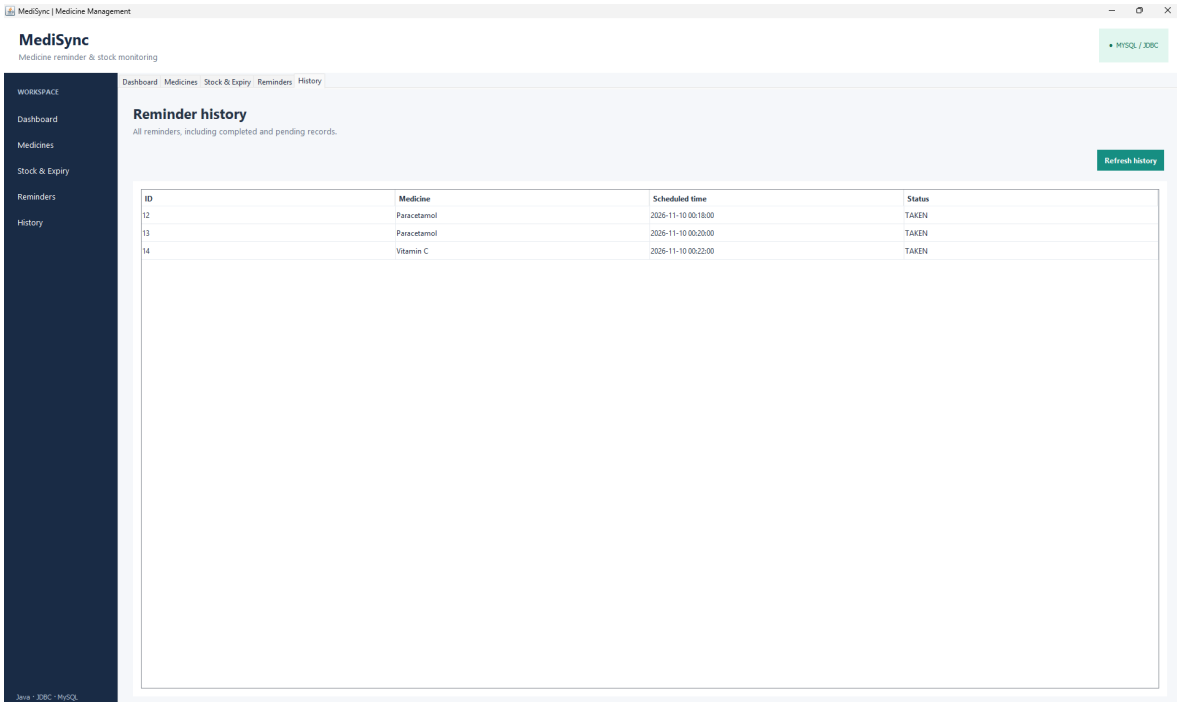


Figure 8.5: Reminder history with three TAKEN outcomes.

9. Database Connectivity Using JDBC

9.1 Database Configuration

The database is named `medicine_system`, uses utf8mb4 and utf8mb4_unicode_ci, and is accessed using JDBC. The table definitions and constraints follow the supplied project schema and MySQL reference documentation [6]. The project connection URL is `jdbc:mysql://localhost:3306/`. Credentials should remain in environment variables and should not be placed in the report.

9.2 Medicines Table

The table contains medicine_id INT primary key auto-increment, name VARCHAR(100), type VARCHAR(50), dosage VARCHAR(100), quantity INT, low_stock_threshold INT, expiry_date DATE, notes VARCHAR(500), and created_at TIMESTAMP. Quantity and threshold have non-negative checks. Indexes are defined on name and expiry_date.

9.3 Reminders Table

The table contains reminder_id INT primary key auto-increment, medicine_id INT foreign key, reminder_time DATETIME, status VARCHAR(20), notified_at nullable DATETIME, completed_at nullable DATETIME, and created_at TIMESTAMP. A check constraint restricts the status values. An index on (status, reminder_time) supports due-reminder lookup.

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

reminder_id	medicine_id	reminder_time	status	notified_at	completed_at	created_at
12	4	2026-11-10 00:18:00	TAKEN	2026-11-10 00:18:01	2026-11-10 00:18:36	2026-11-10 00:17:57
13	4	2026-11-10 00:20:00	TAKEN	NULL	2026-11-10 00:19:21	2026-11-10 00:19:11
14	6	2026-11-10 00:22:00	TAKEN	2026-11-10 00:22:01	2026-11-10 00:22:13	2026-11-10 00:21:19
NULL	NULL	NULL	NULL	NULL	NULL	NULL

Figure 9.1: MySQL Workbench result grid for reminder records.

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

medicine_id	name	type	dosage	quantity	low_stock_threshold	expiry_date	notes	created_at
4	Paracetamol	Tablet	500 mg	50	10	2027-11-10	Commonly used to relieve mild to moderate pain…	2026-11-10 00:05:14
5	Cetirizine	Tablet	10 mg	25	5	2027-08-15	Antihistamine used to relieve allergy symptoms …	2026-11-10 00:06:04
6	Vitamin C	Tablet	500 mg	8	10	2028-03-20	Vitamin supplement that supports normal immun…	2026-11-10 00:07:25
7	ORS	Powder	21.8 g sachet	30	5	2027-06-10	Oral rehydration salts used to help replace fluid…	2026-11-10 00:08:56
8	Pantoprazole	Tablet	40 mg	4	10	2027-04-18	Reduces stomach acid production and is used fo…	2026-11-10 00:10:22
9	Ibuprofen	Tablet	200 mg	20	5	2028-01-12	Non-steroidal anti-inflammatory medicine used t…	2026-11-10 00:11:39
10	Calcium + Vitamin D3	Tablet	500 mg + 250 IU	15	5	2027-09-25	Supplement that supports bone health and nor…	2026-11-10 00:12:32
11	Azithromycin	Tablet	500 mg	3	5	2027-02-10	Antibiotic used to treat certain bacterial infectio…	2026-11-10 00:13:45
12	Multivitamin	Tablet	Standard formulation	40	10	2028-05-15	Supplement containing multiple vitamins to supp…	2026-11-10 00:14:45
13	Antacid	Tablet	Standard formulation	6	10	2027-01-20	Used to neutralize stomach acid and provide reli…	2026-11-10 00:15:34
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Figure 9.2: MySQL Workbench result grid for medicine records.

10. Source Code and Output Screenshots

The main source components are `MediSyncGUI.java`, `Medicine.java`, `Reminder.java`, `MedicineDAO.java`, `ReminderDAO.java` and `DBConnection.java`. The `pom.xml` file defines Maven build configuration and dependencies.

The schema creation script should contain database and table definitions. Diagnostic queries such as `SELECT * FROM medicines` and `SELECT * FROM reminders` are better kept in a separate database testing script.

Build: `mvn clean compile`

Run: `mvn exec:java "-Dexec.mainClass=gui.MediSyncGUI"`

11. Testing and Results

The results below are limited to the supplied application and MySQL screenshots. They are reported as observed evidence rather than as a claim that every possible test case has been executed.

Test case	Expected behaviour	Evidence status
Dashboard summary	Display inventory and reminder summary counts.	Observed: 10 medicines, 4 low-stock, 0 expiring/expired, 0 active reminders.
Medicine inventory	Display stored medicine records and inventory fields.	Observed: 10 records displayed.
Low-stock monitoring	Identify records where quantity is below the configured threshold.	Observed: 4 records labelled LOW STOCK.
Expiry monitoring	Provide filters for expired and expiring records.	Interface observed; no matching expiry alert shown in supplied capture.
Reminder notification	Display a notification when a reminder becomes due.	Observed: Paracetamol notification popup.
Reminder history	Display reminder records and recorded outcomes.	Observed: 3 records with TAKEN status.
MySQL persistence	Retain medicine and reminder records in database tables.	Observed in MySQL Workbench result grids.
MISSED / SKIPPED outcomes	Allow user to record these outcomes.	Not demonstrated in supplied screenshots.
Database failure handling	Present controlled feedback when database access fails.	Not demonstrated in supplied screenshots.
Input validation and delete cascade	Reject invalid data and maintain referential integrity.	Not demonstrated by supplied screenshots.

Table 11.1: Evidence-based functional test summary.

The dashboard shows 10 medicines, 4 low-stock records, 0 expiring/expired records and 0 active reminders at capture time. The inventory view displays ten medicine records. The stock view marks four records LOW STOCK. A Paracetamol due-reminder notification was captured. Reminder history shows three records, all TAKEN. MySQL Workbench result

grids show medicine and reminder rows.

MISSED and SKIPPED outcomes, database failure handling, invalid input handling and delete-cascade behaviour are not demonstrated in the supplied screenshots and are not claimed as tested. One reminder row in the SQL screenshot has a NULL `notified_at`; therefore, the report does not assume all reminder rows followed an identical notification path.

12. Conclusion and Future Enhancements

MediSync demonstrates integration of Java Swing, object-oriented program organisation, JDBC and MySQL in a desktop application for medicine inventory and reminder management. The evidence shows inventory records, low-stock alerts, a due reminder notification, reminder history and corresponding SQL records.

Future enhancements include background scheduling off the Swing Event Dispatch Thread, recurring reminders, MISSED/SKIPPED reporting, authentication, database backup and restore, CSV/PDF export, stronger validation, operating-system notifications and automated unit/integration tests. The application is an inventory and reminder aid, not a substitute for medical advice.

A. Reminder Workflow

Figure A.1 illustrates the reminder lifecycle implemented by the application. The flow represents the reminder process while the desktop application is running.

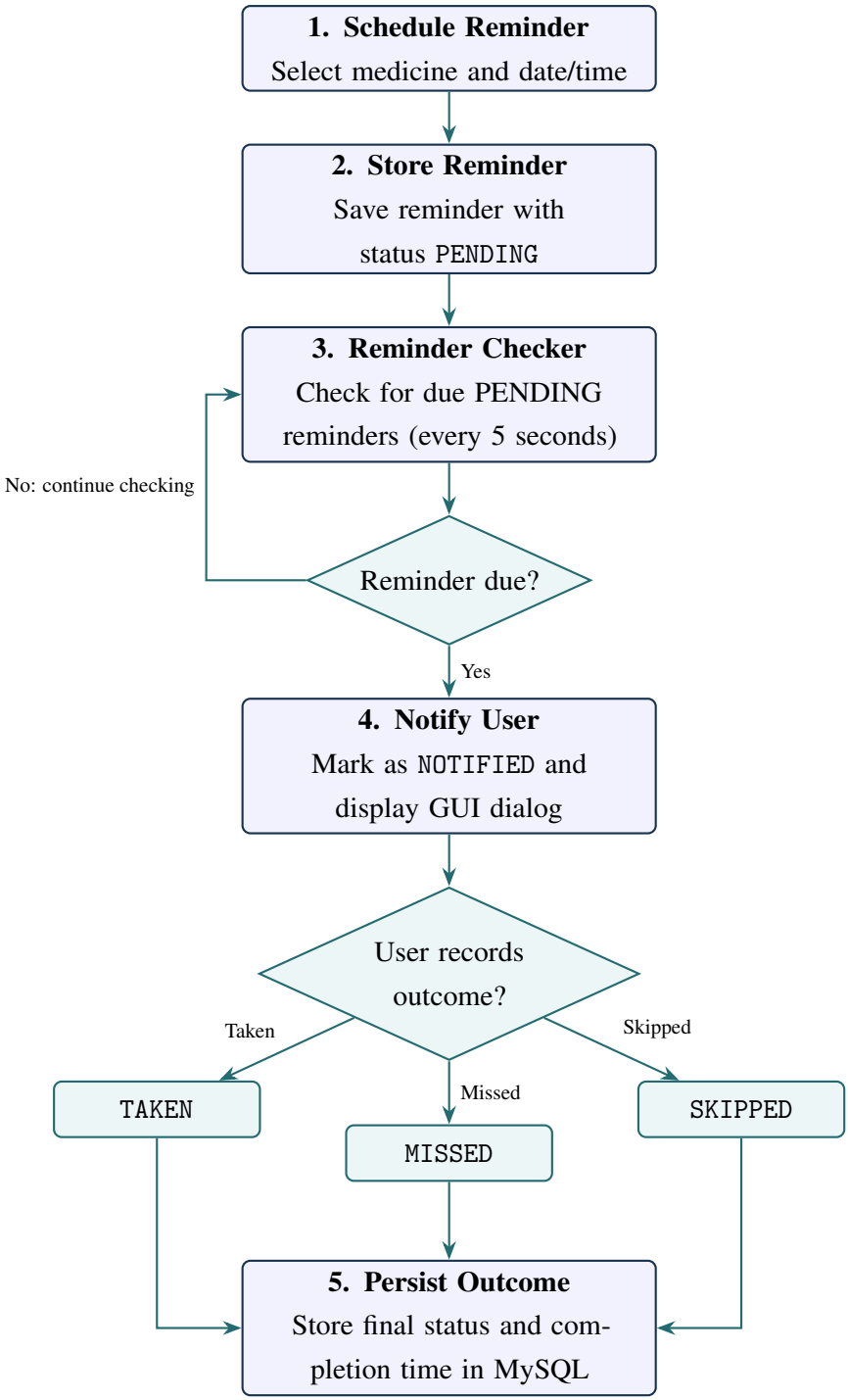


Figure A.1: MediSync reminder workflow from scheduling through notification and user-recorded outcome.

1. The user selects a medicine and schedules a date/time; the record is inserted as PENDING.

2. While the application is running, the reminder checker looks for due PENDING reminders.

3. A due reminder can be marked NOTIFIED and `notified_at` can record the notification

time; the GUI displays a dialog.

4. The user records TAKEN, MISSED or SKIPPED; the outcome and `completed_at` are stored.

The screenshots demonstrate a notification and TAKEN outcomes. MISSED and SKIPPED are allowed by the schema but are not shown in the supplied screenshots.

B. Demonstration Sequence

1. Launch the application after confirming MySQL is running.
2. Explain the Dashboard summary.
3. Show the Medicines table and record actions.
4. Demonstrate Stock & Expiry filters.
5. Schedule a near-future reminder.
6. Wait for the notification while the application remains open.
7. Record an outcome and verify it in History.
8. Run SELECT queries in MySQL Workbench to show persisted records.

Bibliography

[1] Oracle, *Java Platform, Standard Edition Documentation*. Available at: https://docs.oracle.com/en/java/

[2] Oracle, *Creating a GUI With Swing*. Available at: https://docs.oracle.com/javase/tutorial/uiswing/

[3] Oracle, *JDBC Database Access*. Available at: https://docs.oracle.com/javase/tutorial/jdbc/

[4] Oracle, *Java SE API Documentation: javax.swing.Timer*. Available at: https://docs.oracle.com/en/java/javase/17/docs/api/java.desktop/javax/swing/Timer.html

[5] Oracle, *Java SE API Documentation: PreparedStatement*. Available at: https://docs.oracle.com/en/java/javase/17/docs/api/java.sql/java/sql/PreparedStatement.html

[6] MySQL, *MySQL 8.0 Reference Manual*. Available at: https://dev.mysql.com/doc/refman/8.0/en/

[7] MySQL, *MySQL Connector/J Developer Guide*. Available at: https://dev.mysql.com/doc/connector-j/en/

[8] Apache Software Foundation, *Apache Maven Documentation*. Available at: https://maven.apache.org/guides/

[9] MediSync Project Team, *MediSync: Medicine Reminder and Stock Monitoring System*. Project source code, database schema, and implementation screenshots. SRM University–AP, 2026.